Il WorkManager e lo scheduler

Come rsedit esegue il lavoro asincrono: architettura, logica e uso — documento di progetto

Indice

1 In breve 2
1.1 Chi possiede cosa 2
2 L'architettura 3
2.1 Chi parla con chi 3
2.2 Il ciclo di vita del WorkManager 3
2.3 I thread dedicati 3
3 I lavori 5
3.1 Il contratto 5
3.2 Le regole del passo 5
3.3 Gli stati 6
3.4 Le uscite 6
3.5 Nomi e sostituzione 6
4 La logica dello scheduler 7
4.1 Il ciclo dell'esecutore 7
4.2 Chi passa prima 7
4.3 Le scadenze 7
4.4 Gli eventi 7
4.5 Più esecutori 8
5 Misure, segnalazioni, opzioni 9
5.1 Che cosa si misura 9
5.2 Le segnalazioni 9
5.3 Le opzioni 10
6 Usarlo 11
6.1 Da Rust 11
6.2 Da Lisp 12
7 Prove e piano 14
7.1 Come si prova 14
7.2 Il piano 14
8 Appendice 16
8.1 Glossario 16
8.2 Le decisioni 17

1 In breve

rsedit fa molto lavoro lontano dal thread su cui l'utente scrive: colora il testo, prescansiona le espressioni, controlla i file cambiati, salva le copie automatiche, esegue i lavori Lisp, i comandi di shell e le ricerche nei file. Questo documento descrive la struttura che esegue tutto questo lavoro: il *WorkManager* e il suo *scheduler*. Spiega come sono fatti, quale logica seguono e come si usano da Rust e da Lisp. I termini sono riassunti nel glossario in fondo (§ 8.1).

Stato: implementato, e assorbito nel manuale. Le patch del piano (§ 7.2) hanno costruito il WorkManager in `work/`, e `background/` non c'è più. Ciò che di questo documento vale ancora — architettura, lavori, logica dello scheduler, argomenti, misure, opzioni, thread dedicati, watchdog e prove — è nel capitolo «Il lavoro in background» del manuale (`architettura-editor.typ`), descritto sul codice di oggi: il riferimento è quello. Qui restano il piano, l'uso da Lisp, che la pagina `background` spiega a chi lo scrive, e le idee per dopo; gli esempi mostrano la forma prevista prima del codice. Il documento si toglie nella fase D.

Le regole, in sei righe.

1. **Un solo punto d'ingresso.** Tutto il lavoro asincrono passa dal WorkManager, ha un nome e un `WorkId`, e si può elencare e fermare.
2. **Due forme di lavoro.** I lavori a passi girano sugli esecutori dello scheduler; ciò che può bloccarsi (un processo, il disco) ha un thread dedicato, di cui il WorkManager tiene l'handle.
3. **Un passo non aspetta mai.** Fa una quantità limitata di lavoro e dice come proseguire.
4. **Lo scheduler misura e segnala, non interrompe.** Un passo troppo lungo finisce nel log; la correzione si fa a mano.
5. **Gli eventi non portano dati.** Dicono «vai a guardare»: la fonte della verità è lo stato.
6. **Tutto si prova senza thread.** Il nucleo dello scheduler riceve l'ora come parametro.

1.1 Chi possiede cosa

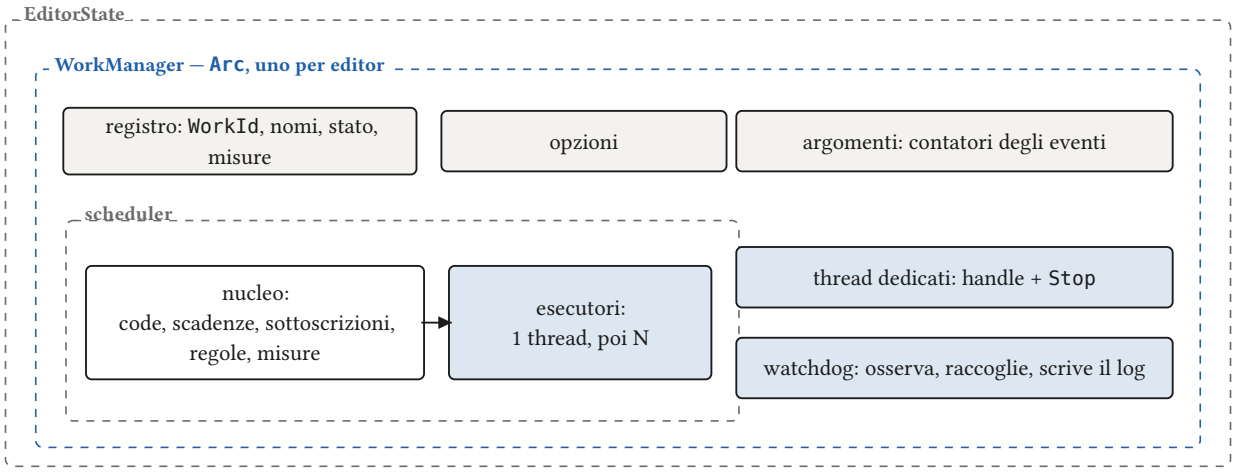


Figura 1: Il WorkManager e le sue parti. In azzurro i thread: tutto il resto sono dati, raggiunti da qualunque thread attraverso il WorkManager.

2 L’architettura

2.1 Chi parla con chi

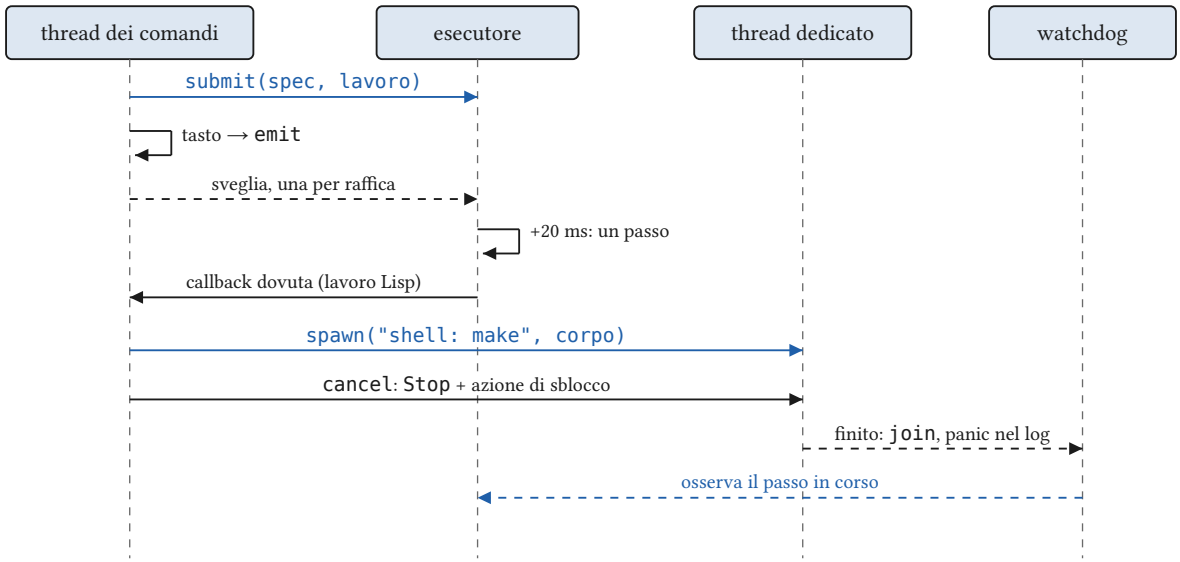


Figura 2: I thread e i messaggi tra loro. Le chiamate al WorkManager (`submit`, `spawn`, `cancel`) avvengono sul thread di chi chiama e non aspettano.

- **Il thread dei comandi** chiama il WorkManager: sottomette lavori, avvia thread dedicati, ferma, cambia opzioni. Le porte delle modifiche emettono gli argomenti, e una raffica di tasti produce una sola sveglia.
- **L’esecutore** esegue i passi uno alla volta. I lavori Lisp non chiamano mai il loro modo: lasciano una callback dovuta, che il thread dei comandi esegue tra un comando e l’altro, come oggi.
- **I thread dedicati** lavorano per conto loro. Si fermano con `Stop` e con un’azione che li sblocca, per esempio uccidere il processo di cui leggono l’output.
- **Il watchdog** osserva gli esecutori, raccoglie i thread dedicati finiti (`join`) e scrive il file di log. Così una segnalazione arriva nel log anche quando ciò che segnala è bloccato.

2.2 Il ciclo di vita del WorkManager

- **Nasce** in `EditorState::new`, prima che `init.lisp` venga valutato: lavori e opzioni si possono impostare dalla configurazione.
- **Ha due modalità.** *Con thread* è l’editor vero. *Manuale* è per i test: nessun esecutore gira da solo, e il test fa avanzare il lavoro quando vuole (§ 7.1).
- **Termina con `shutdown(entro)`**, chiamato dalla TUI all’uscita e, nei test, da un oggetto guardia. Smette di accettare lavoro, fa uscire i lavori con `Exit::Shutdown`, manda `Stop` ai thread dedicati, aspetta fino alla scadenza e abbandona chi resta.

`shutdown` è esplicito per una ragione precisa: i thread tengono una copia di `EditorState`, che contiene il WorkManager. Il ciclo di riferimenti impedisce al `Drop` di bastare. È lo stesso ciclo che oggi tiene in vita per sempre lo scheduler di ogni editor creato, anche nei test.

2.3 I thread dedicati

Un thread dedicato serve a ciò che può bloccarsi e non si può dividere in passi: leggere l’output di un processo, una lettura dal disco. Il WorkManager lo avvia, ne tiene l’handle e lo elenca nel registro come un lavoro.



Figura 3: Il ciclo di vita di un thread dedicato. Il watchdog fa `join` dei thread finiti e scrive nel log un eventuale panic.

- `spawn(nome, corpo)` avvia il thread e restituisce un `WorkId`. Il corpo riceve uno `Stop`.
- **Un thread non si uccide.** Rust non ha un modo sicuro per farlo: un thread ucciso mentre tiene il lock di un buffer bloccherebbe l'editor al primo accesso. `cancel` invece alza il segnale di `Stop`, che il corpo controlla nei punti sicuri, ed esegue l'azione registrata con `stop.on_cancel`, che sblocca ciò che il thread aspetta: per un comando di shell, uccidere il processo chiude le sue pipe.
- **Il `join` non blocca nessuno.** Il watchdog lo fa solo sui thread che risultano finiti (`is_finished`). Un panic nel corpo torna dal `join` e va nel log.
- **Un thread che non si ferma**, come un `stat` su un disco di rete appeso, resta elencato «in chiusura», viene segnalato e, allo `shutdown`, abbandonato.

3 I lavori

3.1 Il contratto

```rust
/// Un lavoro a passi. Lo scheduler chiama `step`; il lavoro fa una quantità
/// limitata di lavoro e dice come proseguire.
pub trait Job<B: BufferTrait>: Send + 'static {
    fn step(&mut self, cx: &mut StepContext<'_, B>) -> Result<Step, JobError>;

    /// Chiamata una volta sola, quando il lavoro esce, per qualunque motivo.
    fn exit(&mut self, editor: &EditorState<B>, why: Exit) {}
}

pub enum Step {
    Continue,              // c'è altro da fare: rimettimi in coda
    Sleep(Duration),       // svegliami tra un po'
    Wait,                  // svegliami a un mio evento
    WaitAtMost(Duration),  // a un mio evento, o al più tardi tra un po'
    Done,                  // ho finito
}

pub enum Exit { Finished, Failed, Panicked, Cancelled, Shutdown }

pub struct StepContext<'a, B: BufferTrait> {
    pub editor: &'a EditorState<B>,
    pub woken_by: Wake,    // Start, Turn, Event, Timer
    // …
}

impl<B: BufferTrait> StepContext<'_, B> {
    /// Quanto ha fatto il passo, e su che cosa: la misura deterministica.
    pub fn worked(&mut self, units: u64, on: &str) { /* … */ }
}
```

Un lavoro si sottomette con una `Spec`, che ne dichiara il nome e le proprietà:

```rust
let id: WorkId = work.submit(
    Spec::new("highlighter")
        .class(Class::High)           // il default è Class::Normal
        .unit("righe")                // l'unità di cx.worked
        .on("buffer-changed"),        // un argomento a cui è sottoscritto
    Box::new(Highlighter),
);
```

3.2 Le regole del passo

- **Non aspetta mai.** Niente I/O che può bloccarsi, che va su un thread dedicato; niente `sleep`. Chi deve aspettare lo dice nell'esito.
- **Non tiene lock da un passo all'altro.** Copia sotto lock, lavora senza, salva solo se la versione non è cambiata: la regola di oggi.
- **Non guarda l'orologio per decidere quanto fare.** Divide il lavoro in unità dei suoi dati (righe, file, passi di carburante) e lo dichiara con `cx.worked`. Il tempo lo misura lo scheduler.
- **Non si interrompe.** `cancel` ha effetto alla fine del passo in corso.

- **Gli errori passano dal `Result`.** Lo scheduler li scrive nel log con il nome del lavoro e chiama `exit(Failed)`.

3.3 Gli stati

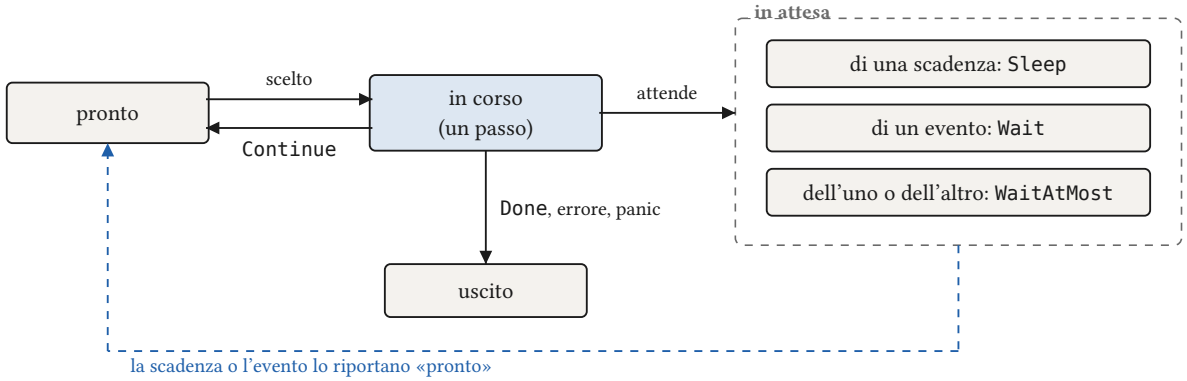


Figura 4: Gli stati di un lavoro. `cancel` e `shutdown` portano a «uscito» da qualunque stato; se il lavoro è in corso, alla fine del passo.

3.4 Le uscite

`Exit`	Quando
`Finished`	il passo ha risposto `Done`
`Failed`	`step` ha restituito un errore
`Panicked`	il passo è andato in panic: lo scheduler lo intercetta con `catch_unwind`
`Cancelled`	qualcuno l'ha fermato, o un lavoro con lo stesso nome l'ha sostituito
`Shutdown`	l'editor si chiude

In `exit` va la pulizia: un contatore da scalare, la callback `on-done` di un lavoro Lisp. Oggi questo codice è sparso nei rami di `execute`.

3.5 Nomi e sostituzione

Il nome lo dà chi sottomette, e lo tiene il registro. I nomi sono unici: sottomettere un nome già presente fa uscire il vecchio lavoro con `Cancelled` e mette in coda il nuovo. È la regola di oggi per i lavori Lisp, estesa a tutti. Il `WorkId` prende il posto delle generazioni che oggi stanno in `Runtime`.

4 La logica dello scheduler

4.1 Il ciclo dell'esecutore

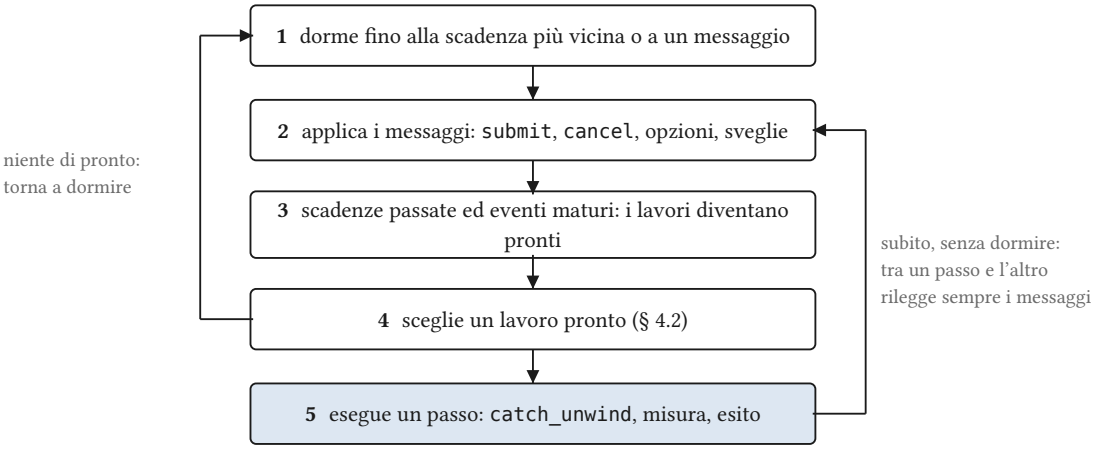


Figura 5: Il ciclo di un esecutore. Il nucleo decide i punti 2–4 senza leggere l'orologio: l'ora gli arriva come parametro.

4.2 Chi passa prima

- **Due classi.** `High` è per ciò che l'utente sta aspettando sullo schermo: evidenziatore e prescanner. `Normal` è il default: controllo dei file, salvataggio automatico, tutti i lavori Lisp.
- **Una coda per classe, a turno.** Un lavoro entra in fondo alla coda della sua classe ogni volta che diventa pronto: sottomesso, `Continue`, scadenza passata, evento. Se è l'unico pronto, riparte subito.
- **Prima la classe alta, ma non sempre.** Dopo `high-steps` passi alti di fila (4 per default), se un lavoro normale è pronto tocca a lui. La regola si conta in passi, non in tempo: è deterministica e si prova senza orologio.



Figura 6: La regola con `high-steps = 4`. Quando l'evidenziatore ha finito, l'indice ha l'esecutore tutto per sé.

4.3 Le scadenze

- **Un lavoro ha al più una scadenza**, messa da `Sleep` o `WaitAtMost`. Chi ne ha più d'una, come il salvataggio automatico (una per buffer), le tiene per sé e chiede ogni volta la più vicina.
- **La scadenza si conta dalla fine del passo.** Un lavoro lento non si accoda a sé stesso, e un periodo saltato non si recupera.
- **Il ritardo con cui un lavoro parte** rispetto alla scadenza viene misurato e, oltre la soglia, segnalato.

4.4 Gli eventi

- **Un argomento è un contatore.** Emetterlo costa un incremento atomico, più una sveglia all'esecutore solo se non ne ha già una in arrivo: si può fare anche dentro il lock di un buffer.
- **La sottoscrizione è persistente.** Un lavoro ha eventi in sospeso se il contatore di un suo argomento ha superato l'ultimo valore che ha visto. Quel valore si aggiorna *prima* che il passo parta: un evento che arriva durante il passo non va perso.
- **Il ritardo raccoglie le raffiche.** Un lavoro diventa pronto `event-delay` dopo il primo evento non ancora visto (20 ms per default), e gli eventi che arrivano nel frattempo si sommano a quello.

- **Un `Wait` senza argomenti sottoscritti è un errore.** Il lavoro verrebbe dimenticato, quindi lo scheduler lo segnala.

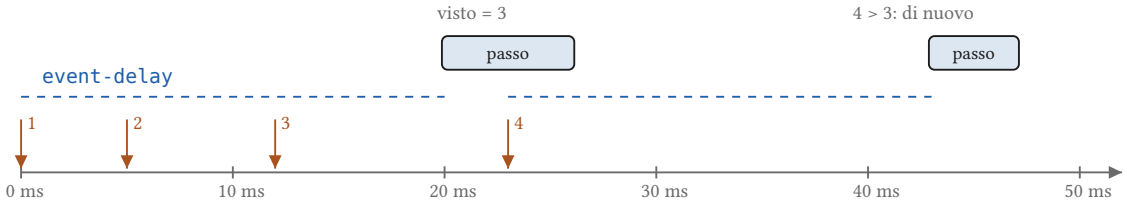


Figura 7: Tre tasti in 12 ms producono un solo passo. Le frecce sono le emissioni, con il valore del contatore. Il quarto tasto arriva durante il passo: il contatore supera il valore visto, e dopo il ritardo il lavoro riparte.

Gli argomenti iniziali sono due:

Argomento	Chi lo emette
`buffer-changed`	`edits::changed` (le porte, undo, redo, `replace-back`), `adopt_text`, `change_major_mode` e la creazione di un buffer
`syntax-changed`	chi cambia una grammatica o una tabella di sintassi. Il contatore `grammar_epoch` che esiste già è un argomento di questo tipo.

Il modulo dei buffer non conosce lo scheduler: alla creazione, ogni buffer riceve l'argomento `buffer-changed` già risolto, cioè un contatore con la sua sveglia.

Gli eventi non portano dati, e non è una dimenticanza. Un messaggio con dati va consegnato uno per uno, invecchia (la riga indicata vale per una versione già superata) e, se una coda si riempie, va perso: non può essere la fonte della verità. Ciò che serve a un lavoro si registra nello stato, dove nasce. Per esempio, la geometria di una modifica (riga, righe aggiunte o tolte) andrà nelle cache dell'evidenziatore e del prescanner, per smettere di ricolorare appena lo stato del lexer torna uguale a prima. I dati che non esistono altrove, come l'esito di una lettura, viaggeranno in una casella di posta per lavoro, insieme all'I/O asincrono dal Lisp.

4.5 Più esecutori

Ogni esecutore ha le sue code e il suo conteggio dei passi alti. Un lavoro viene assegnato all'esecutore meno carico quando entra e ci resta: l'interprete ha uno stato per thread (carburante, dati di match, `in_worker`), e una fiber ripresa su un altro thread lo troverebbe diverso. Un sostituto con lo stesso nome va sullo stesso esecutore del vecchio, così i due non girano mai insieme. Il numero di esecutori non compare nell'interfaccia.

5 Misure, segnalazioni, opzioni

5.1 Che cosa si misura

- **Ogni passo:** lavoro, classe, esecutore, inizio, durata, attesa in coda, ritardo sulla scadenza, unità dichiarate, esito. Per i lavori Lisp le unità sono i passi di carburante, contati dallo scheduler.
- **Ogni lavoro:** passi, tempo totale, passo più lungo e su che cosa, stato.
- **Ogni esecutore:** occupazione nell'ultimo secondo e passo in corso.
- **Ogni thread dedicato:** avvio, stato, durata.
- **Le callback dovute,** sul thread dei comandi: è lì che una callback lenta blocca l'interfaccia.

Il costo per passo è di due letture dell'orologio e qualche somma; una segnalazione si compone solo quando una regola scatta.

5.2 Le segnalazioni

Segnalazione	Quando (opzione)	Cosa riporta
Passo lungo	un passo dura più di 40 ms (`long-step`)	lavoro, durata, unità, costo per unità, su che cosa
Ritardo	un lavoro pronto aspetta più di 40 ms se alto, 200 ms se normale (`late-high`, `late-normal`)	di quanto, e chi ha occupato l'esecutore
Carico	un esecutore occupato per più di metà dell'ultimo secondo (`busy-share`)	la percentuale e i lavori che la causano
Passo bloccato	lo stesso passo in corso da più di 1 s, poi a 2 s, 4 s… (`stuck-step`)	lavoro e tempo; la scrive il watchdog
Panic, errore	sempre	lavoro e messaggio; il lavoro esce
Thread che non si ferma	vivo 1 s dopo `cancel`	nome e tempo trascorso
Callback lenta	una callback dovuta oltre 40 ms	lavoro e durata

La soglia del ritardo è diversa per le due classi perché un lavoro normale, per progetto, aspetta fino a `high-steps` passi alti. Per ogni coppia lavoro e regola la prima occorrenza va subito nel log; le successive confluiscono in un riepilogo, al più uno ogni `summary-every`. Nessuna segnalazione cambia il comportamento di un lavoro.

```
[BKG] passo lungo: highlighter, 912 ms su app.min.js (3 righe, 304 ms/riga)
[BKG] ritardo: file-watcher partito 230 ms tardi, dietro highlighter (6 passi)
[BKG] carico: 64% dell'ultimo secondo (highlighter 41%, manpage-index 20%)
[BKG] passo bloccato: manpage-index in corso da 2,0 s
[BKG] thread: shell «make» ancora vivo 1,0 s dopo la richiesta di stop
[BKG] riepilogo: highlighter, 12 passi lunghi negli ultimi 10 s (max 912 ms)
```

5.3 Le opzioni

Opzione	Default	Cosa regola
`high-steps`	4	passi alti di fila prima di uno normale
`event-delay`	20 ms	ritardo dopo un evento
`long-step`	40 ms	soglia del passo lungo
`late-high`, `late-normal`	40, 200 ms	soglie del ritardo
`stuck-step`	1000 ms	soglia del passo bloccato
`busy-share`	50 %	soglia del carico
`summary-every`	10 000 ms	intervallo tra due riepiloghi
`watchdog-every`	250 ms	ogni quanto si sveglia il watchdog

Le soglie partono dal turno di 40 ms, la cadenza a cui oggi girano evidenziatore e renderer. Le opzioni stanno nel WorkManager e si leggono con operazioni atomiche: una modifica vale dal passo successivo.

6 Usarlo

6.1 Da Rust

6.1.1 Un lavoro guidato da eventi

```
/// Colora, un pezzo alla volta, il buffer che ne ha più bisogno.
struct Highlighter;

impl<B: BufferTrait> Job<B> for Highlighter {
    fn step(&mut self, cx: &mut StepContext<'_, B>) -> Result<Step, JobError> {
        match cx.editor.highlight_one_turn() {
            // Ha colorato delle righe: forse ne restano.
            Some((buffer, lines)) => {
                cx.worked(lines as u64, &buffer);
                Ok(Step::Continue)
            }
            // Nessun buffer indietro: dorme fino al prossimo cambiamento.
            None => Ok(Step::Wait),
        }
    }
}

work.submit(
    Spec::new("highlighter").class(Class::High).unit("righe")
        .on("buffer-changed").on("syntax-changed"),
    Box::new(Highlighter),
);
```

6.1.2 Un lavoro periodico

```
/// Controlla alcuni file per volta, per sempre.
struct FileWatcher<B: BufferTrait> { next: usize, env: Arc<Env<EditorState<B>>> }

impl<B: BufferTrait> Job<B> for FileWatcher<B> {
    fn step(&mut self, cx: &mut StepContext<'_, B>) -> Result<Step, JobError> {
        if watching(&self.env) {
            let (next, checked) = cx.editor.watch_files_turn(self.next);
            self.next = next;
            cx.worked(checked as u64, "file");
        }
        // Riletto a ogni passo: un (setq watch-file-interval 10) vale subito.
        Ok(Step::Sleep(watch_interval(&self.env)))
    }
}
```

Rileggere l'intervallo a ogni passo corregge anche un difetto di oggi: `FileWatcher` e `AutoSaver` vengono programmati prima che `init.lisp` sia valutato, quindi `watch-file-interval` e `auto-save-interval` impostati lì non hanno mai effetto.

6.1.3 Un thread dedicato

```
let child = Arc::new(Mutex::new(command.spawn()?));
let stdout = child.lock().unwrap().stdout.take().unwrap();
let killer = child.clone();

let id = work.spawn("shell: make", move |stop: Stop| {
    // Uccidere il processo chiude la pipe: è ciò che sblocca la lettura.
    stop.on_cancel(move || { let _ = killer.lock().unwrap().kill(); });
    for line in BufReader::new(stdout).lines() {
        if stop.is_cancelled() { break; }
        // … accoda la riga al buffer, attraverso una porta
    }
});

work.cancel(id);   // più tardi, da un comando
```

6.1.4 Emettere un argomento

```
// In edits::changed, sotto il lock del buffer: un incremento atomico.
buf.changed.emit();

// Altrove, un argomento si risolve per nome una volta sola:
let syntax = work.topic("syntax-changed");
syntax.emit();
```

6.2 Da Lisp

6.2.1 Worker e `yield`

`define-worker` e `background-call` mantengono la forma di oggi. L'intervallo facoltativo diventa l'attesa di default, cioè ciò che vale quando la fiber scrive solo `(yield)`.

```
(yield)                                    ; attesa di default: 40 ms se omessa
(yield 'continue)                          ; Continue: riparti appena puoi
(yield '(sleep 250))                       ; Sleep: tra 250 ms
(yield '(wait buffer-changed))             ; Wait: al prossimo evento indicato
(yield '(wait-at-most 500 buffer-changed)) ; WaitAtMost: evento o 500 ms
(yield 42)                                 ; altro valore: come (yield)
```

Gli argomenti dell'ultima attesa diventano la sottoscrizione del lavoro e restano validi finché non ne indica altri. L'ultima riga serve alla compatibilità: chi usa il valore di `yield` per comunicare l'avanzamento non cambia comportamento.

```
;; Indicizza a pezzi; quando ha finito, dorme fino alla prossima modifica.
(define-worker 'symbol-index
  (fiber
    (while t
      (cond ((index-stale-p)
             (index-one-chunk)
             (yield 'continue))                ; c'è altro: riparti subito
            (t
             (yield '(wait buffer-changed)))))))  ; niente da fare: aspetta
```

6.2.2 `background-call`

```
(background-call 'manpage-index
  (fiber (let ((left manpage-path))
           (while left
             (manpage--index-one (car left))
             (setq left (cdr left))
             (yield 'continue))))     ; un pezzo per volta, senza aspettare 40 ms
  'manpage--index-progress            ; dopo ogni yield, sul thread dei comandi
  'manpage--index-done)               ; alla fine, secondo l'uscita
```

Uscita	`on-done`
`Finished`	chiamata con `(NOME t)`
`Failed`, `Panicked`	chiamata con `(NOME nil)`
`Cancelled`, `Shutdown`	non chiamata: chi ha fermato il lavoro lo sa già

6.2.3 Eventi propri, opzioni e misure

```
;; Un modo avvisa il suo worker senza aspettare una modifica.
(emit-event 'my-mode-refresh)
(define-worker 'my-mode-indexer
  (fiber (while t
           (my-mode-reindex)
           (yield '(wait my-mode-refresh buffer-changed)))))

(set-work-option 'high-steps 6)     ; anche da init.lisp
(work-option 'event-delay)          ; → 20
(work-list)                         ; lavori e thread con stato e misure
;; → ((name "highlighter" kind job class high state waiting
;;     steps 1204 total-ms 412 max-ms 38) …)
```

`stop-worker` ferma un lavoro o un thread per nome; `running-workers` diventa un filtro di `work-list`.

7 Prove e piano

7.1 Come si prova

Livello	Come
Il nucleo	Senza thread, con un orologio finto e lavori finti. È il grosso dei test: un minuto di scheduler simulato richiede pochi microsecondi.
WorkManager, esecutori	Pochi test con thread veri. Aspettano una condizione con una scadenza, mai con attese fisse; usano opzioni piccole e lavori che si bloccano su un canale controllato dal test.
Editor e Lisp	Il WorkManager in modalità manuale, con `run_until_idle` e `run_for(durata)`.

Nei test l'editor nasce in modalità manuale: i lavori girano solo quando il test lo chiede, e i thread veri solo dove un test li vuole. Spariscono così sia le corse (oggi `colour_fully` colora a mano mentre lo scheduler vero colora lo stesso editor ogni 40 ms) sia i thread che restano vivi dopo ogni test.

```
#[test]
fn a_burst_of_edits_wakes_the_job_once() {
    let work = WorkManager::manual();                    // niente thread
    let probe = ScriptedJob::new([Step::Wait, Step::Wait]);
    let wakes = probe.wakes();                           // registra i risvegli
    work.submit(Spec::new("probe").on("buffer-changed"), Box::new(probe));
    work.run_until_idle(&editor);                        // primo passo: Wait

    let changed = work.topic("buffer-changed");
    for _ in 0..3 {
        changed.emit();
        work.advance(Duration::from_millis(5));          // tre tasti in 15 ms
    }
    work.run_for(&editor, Duration::from_millis(20));    // il ritardo scade
    assert_eq!(*wakes.lock().unwrap(), [Wake::Start, Wake::Event]);
}
```

Gli altri strumenti:

- **Lavori finti:** `ScriptedJob` risponde una sequenza di esiti e registra i risvegli; `PanicJob` va in panic; `BlockingJob` resta fermo finché il test non lo sblocca.
- **Invarianti su sequenze casuali:** un test genera sottomissioni, `cancel`, emissioni ed esiti con un generatore scritto nel test, e controlla dopo ogni operazione che ogni lavoro stia in un solo posto, che `exit` arrivi una volta sola e che la regola dei `high-steps` valga. Il seme viene stampato, per riprodurre un fallimento.
- **Perdite di thread:** dopo `shutdown()` il numero di thread del processo torna quello di partenza (su Linux, leggendo `/proc`).

7.2 Il piano

Il nuovo modulo `work/` nasce accanto a `background/`, e i lavori ci passano uno alla volta. Ogni passo è una patch che lascia l'editor funzionante e i test verdi.

	Contenuto	Chiude
0	Il log non va più in panic e ha un limite (la patch già pronta).	AVV-3, AVV-4
1	Il nucleo: `Job`, esiti, code, regola delle classi, scadenze, argomenti, misure e segnalazioni, con l'orologio finto e i test, compresi gli invarianti.	
2	Il WorkManager: esecutore, modalità manuale e con thread, registro, opzioni, `shutdown`, test sulle perdite di thread. Nessun lavoro spostato.	BKG-1
3	Gli argomenti alla fonte: `buffer-changed`, `syntax-changed`.	
4	Evidenziatore e prescanner nel WorkManager, guidati da eventi; i test di colorazione con `run_until_idle`.	
5	Controllo dei file e salvataggio automatico, con gli intervalli riletti a ogni passo.	difetto degli intervalli
6	I lavori Lisp: `define-worker`, `background-call`, la tabella di `yield`, `emit-event`, opzioni, `work-list`.	
7	Comandi di shell e ricerca nei file come thread dedicati, con `cancel`.	SHL-1, SHL-2, SHL-3
8	Il watchdog, con la scrittura del log.	
9	Via `background/`: `BackgroundScheduler`, `RunNow` e i vecchi trait.	

Dopo: il resync dell'evidenziatore e del prescanner; la casella di posta e l'I/O asincrono dal Lisp; l'handle restituito da `spawn` in `risp`; la misura dei lock lunghi; più esecutori.

8 Appendice

8.1 Glossario

Termine	Che cos'è
WorkManager	La struttura che possiede tutto il lavoro asincrono: registro, opzioni, argomenti, scheduler, thread dedicati, watchdog. Uno per editor, condiviso con `Arc`.
Lavoro	Qualcosa che lo scheduler esegue a passi. Implementa il trait `Job`.
Passo	Una chiamata a `Job::step`: una quantità limitata di lavoro.
Esito	La risposta di un passo: `Continue`, `Sleep`, `Wait`, `WaitAtMost`, `Done`.
Uscita	La fine di un lavoro, con il suo motivo (`Exit`). Arriva una volta sola.
Classe	`High` o `Normal`: decide chi passa prima.
Esecutore	Un thread dello scheduler. All'inizio uno solo; un lavoro resta sempre sullo stesso.
Nucleo	Lo stato dello scheduler: code, scadenze, sottoscrizioni, regole, misure. Non ha thread né orologio.
Argomento	Un evento con un nome: un contatore che cresce a ogni emissione.
Thread dedicato	Un thread per un lavoro che può bloccarsi, con handle e segnale di stop (`Stop`).
`WorkId`	Il numero di un lavoro o di un thread dedicato. Non si riusa mai.
Opzione	Un valore che regola lo scheduler, modificabile da Lisp mentre l'editor gira.
Segnalazione	Una riga `[BKG]` nel log, quando una misura supera la sua soglia.

8.2 Le decisioni

Tema	Decisione
Struttura	Il WorkManager possiede thread dedicati, registro e opzioni, e contiene lo scheduler, che esegue solo lavori a passi e non crea thread altrui.
Lavoro	Il trait `Job`. Il nome lo dà il registro; nomi unici, il nuovo sostituisce il vecchio; il `WorkId` non si riusa.
Esiti	`Continue`, `Sleep`, `Wait`, `WaitAtMost`, `Done`; errori nel `Result`; `exit` una volta sola, con il motivo.
Partenza	Subito, salvo un ritardo esplicito.
`yield`	Generico, con un'attesa di default quando la fiber non chiede niente.
`on-done`	Secondo il motivo dell'uscita.
Priorità	Due classi, a turno; un passo normale ogni `high-steps` alti, opzione modificabile a runtime; i lavori Lisp sono normali.
Scadenze	Una per lavoro, contata dalla fine del passo.
Eventi	Argomenti con nome, senza dati, letti tramite contatori; sottoscrizione persistente; ritardo dal primo evento non visto; argomenti iniziali `buffer-changed` e `syntax-changed`.
Dati	Non negli argomenti. La geometria delle modifiche va nelle cache (resync); i messaggi con dati in una casella per lavoro, con l'I/O dal Lisp.
Misure	Per passo, lavoro, esecutore, thread dedicato e callback; soglie come opzioni; prima occorrenza subito, poi riepiloghi; `work-list`.
Opzioni	Primitive `work-option`, `set-work-option`, `work-options`.
Prove	Tre livelli; modalità manuale nei test; invarianti su sequenze casuali; test sulle perdite di thread.
Restano valide	Gli sforamenti si segnalano e basta; niente budget di tempo nei lavori; il turno di 40 ms come riferimento; yield esplicito nel Lisp; segnalazioni nel log; sola libreria standard; `spawn` resta nell'interprete.